**Технологии конструирования программного обеспечения**

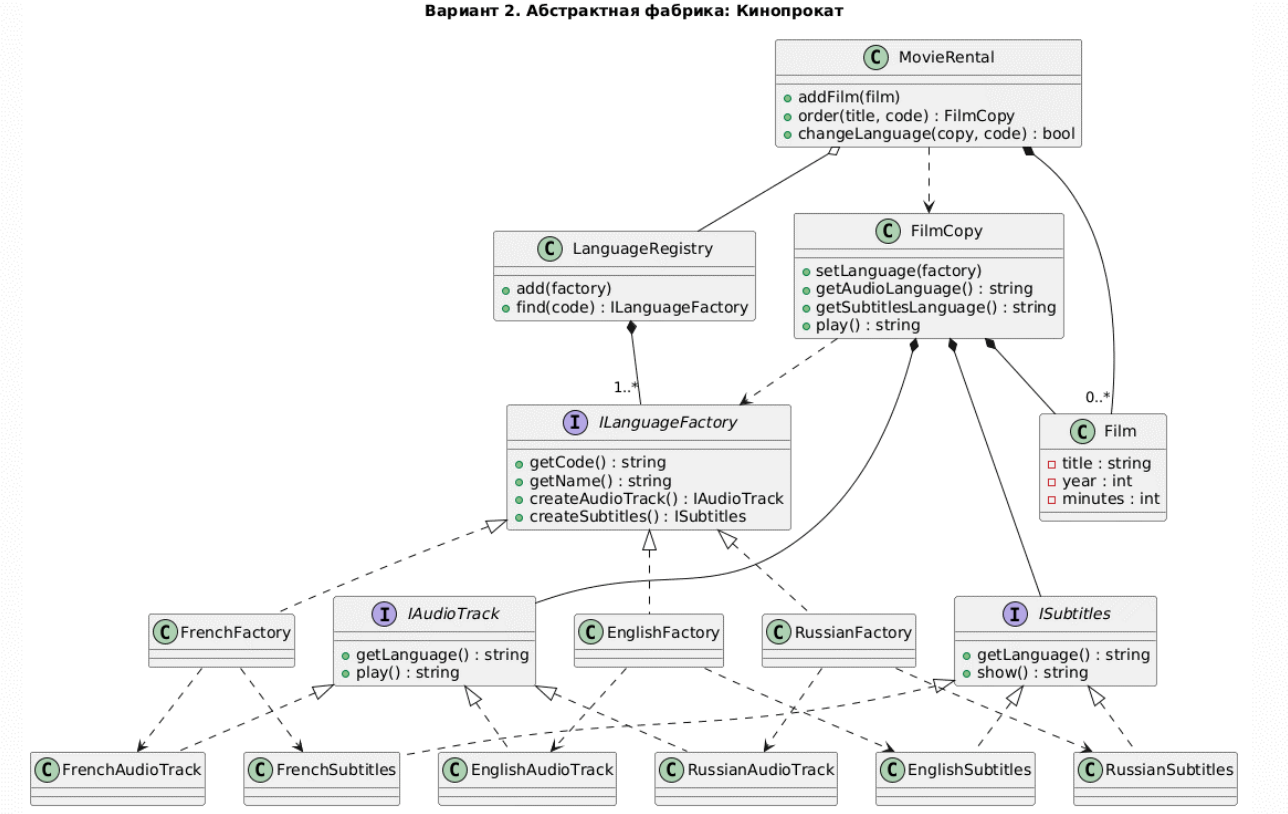
**Отчет по лабораторной работе № 3.2**

Группа: 241-327 Студент: Дубина Анастасия Александровна

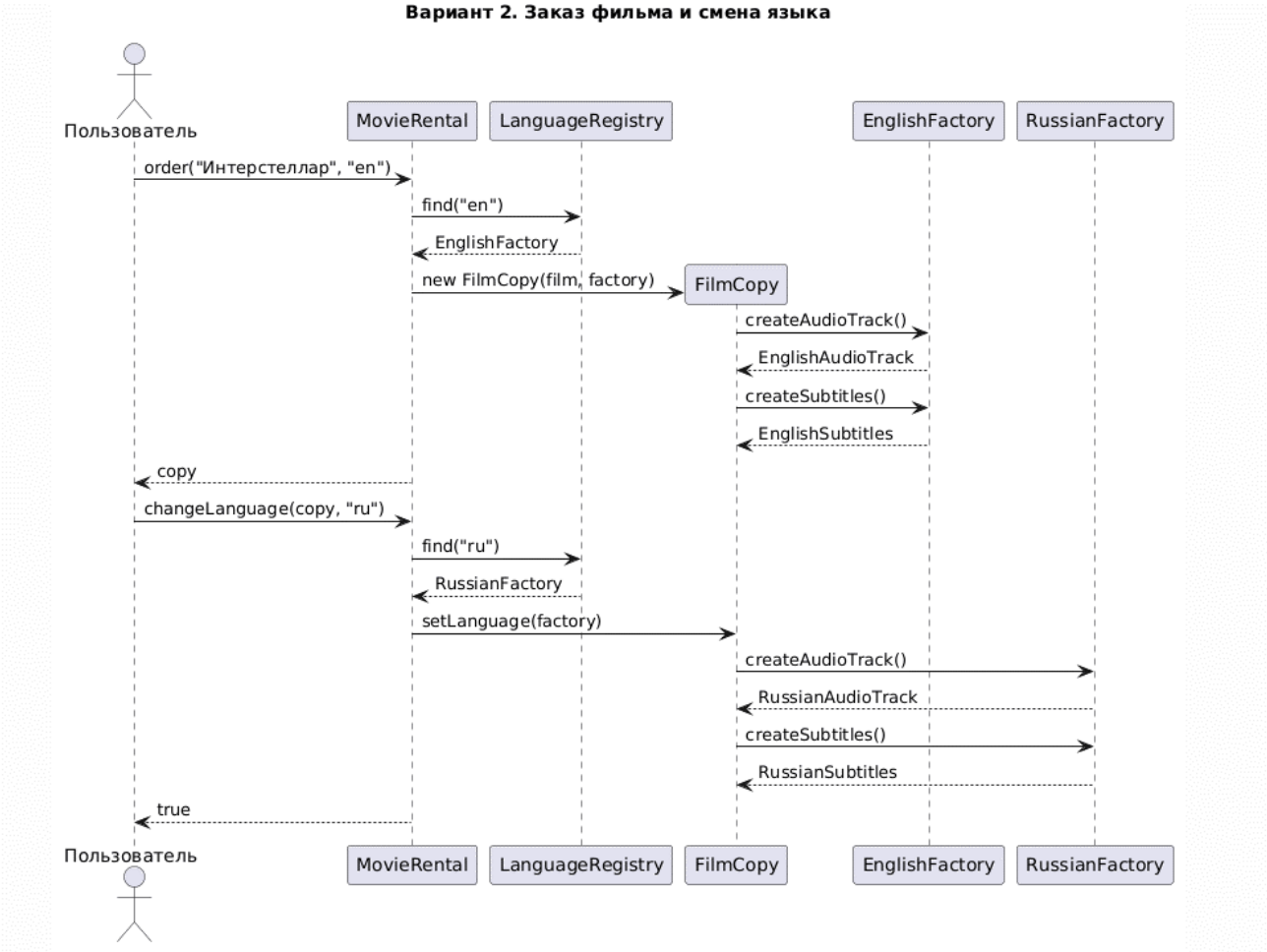
## Задание на лабораторную работу

2. Паттерн Abstract Factory. Разработать систему Кинопрокат. Пользователь может выбрать определённую киноленту, при заказе киноленты указывается язык звуковой дорожки, который совпадает с языком файла субтитров. Система должна поставлять фильм с требуемыми характеристиками, причём при смене языка звуковой дорожки должен меняться и язык файла субтитров и наоборот.

## Диаграмма классов




## Диаграмма последовательности




# Исходный код программы

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <memory>
#include <clocale>

using namespace std;

// ======================== Абстрактные продукты ========================

class IAudioTrack {
public:
   virtual ~IAudioTrack() {}
   virtual string getLanguage() const = 0;
   virtual string play() const = 0;
};
```

```
class ISubtitles {
public:
   virtual ~ISubtitles() {}
   virtual string getLanguage() const = 0;
   virtual string show() const = 0;
};

// ======================== Конкретные продукты ========================

class RussianAudioTrack : public IAudioTrack {
public:
   string getLanguage() const override { return "ru"; }
   string play() const override { return "Звук: русский дубляж"; }
};

class RussianSubtitles : public ISubtitles {
public:
   string getLanguage() const override { return "ru"; }
   string show() const override { return "Субтитры: русские"; }
};

class EnglishAudioTrack : public IAudioTrack {
public:
   string getLanguage() const override { return "en"; }
   string play() const override { return "Звук: английская оригинальная дорожка"; }
};

class EnglishSubtitles : public ISubtitles {
public:
   string getLanguage() const override { return "en"; }
   string show() const override { return "Субтитры: английские"; }
};

class FrenchAudioTrack : public IAudioTrack {
public:
   string getLanguage() const override { return "fr"; }
   string play() const override { return "Звук: французский дубляж"; }
};

class FrenchSubtitles : public ISubtitles {
public:
   string getLanguage() const override { return "fr"; }
   string show() const override { return "Субтитры: французские"; }
};

// ======================== Абстрактная фабрика ========================
```

```
// Фабрика создает семейство согласованных объектов: дорожку и субтитры одного языка.
class ILanguageFactory {
public:
   virtual ~ILanguageFactory() {}
   virtual string getCode() const = 0;
   virtual string getName() const = 0;
   virtual unique_ptr<IAudioTrack> createAudioTrack() const = 0;
   virtual unique_ptr<ISubtitles> createSubtitles() const = 0;
};

// ======================== Конкретные фабрики ========================

class RussianFactory : public ILanguageFactory {
public:
   string getCode() const override { return "ru"; }
   string getName() const override { return "русский"; }
   unique_ptr<IAudioTrack> createAudioTrack() const override {
      return unique_ptr<IAudioTrack>(new RussianAudioTrack());
   }
   unique_ptr<ISubtitles> createSubtitles() const override {
      return unique_ptr<ISubtitles>(new RussianSubtitles());
   }
};

class EnglishFactory : public ILanguageFactory {
public:
   string getCode() const override { return "en"; }
   string getName() const override { return "английский"; }
   unique_ptr<IAudioTrack> createAudioTrack() const override {
      return unique_ptr<IAudioTrack>(new EnglishAudioTrack());
   }
   unique_ptr<ISubtitles> createSubtitles() const override {
      return unique_ptr<ISubtitles>(new EnglishSubtitles());
   }
};

class FrenchFactory : public ILanguageFactory {
public:
   string getCode() const override { return "fr"; }
   string getName() const override { return "французский"; }
   unique_ptr<IAudioTrack> createAudioTrack() const override {
      return unique_ptr<IAudioTrack>(new FrenchAudioTrack());
   }
   unique_ptr<ISubtitles> createSubtitles() const override {
      return unique_ptr<ISubtitles>(new FrenchSubtitles());
   }
```

```
};

// ======================== Предметная область ========================

// Кинолента из каталога
class Film {
public:
   Film(const string& title, int year, int minutes)
      : title(title), year(year), minutes(minutes) {}

   string getTitle() const { return title; }
   int getYear() const { return year; }
   int getMinutes() const { return minutes; }

private:
   string title;
   int year;
   int minutes;
};

// Выданная пользователю копия фильма (клиент абстрактной фабрики).
// Дорожка и субтитры всегда создаются одной фабрикой, поэтому их языки совпадают.
// Отдельно поменять только дорожку или только субтитры нельзя.
class FilmCopy {
public:
   FilmCopy(const Film& film, const ILanguageFactory& factory) : film(film) {
      setLanguage(factory);
   }

   void setLanguage(const ILanguageFactory& factory) {
      audio = factory.createAudioTrack();
      subtitles = factory.createSubtitles();
   }

   const Film& getFilm() const { return film; }
   string getAudioLanguage() const { return audio->getLanguage(); }
   string getSubtitlesLanguage() const { return subtitles->getLanguage(); }

   string play() const {
      return "\"" + film.getTitle() + "\" (" + to_string(film.getYear()) + ", " +
           to_string(film.getMinutes()) + " мин) | " + audio->play() + " | " +
           subtitles->show();
   }

private:
   Film film;
   unique_ptr<IAudioTrack> audio;
```

```
    unique_ptr<ISubtitles> subtitles;
};

// Набор доступных языков: по коду языка выдает нужную фабрику
class LanguageRegistry {
public:
    void add(unique_ptr<ILanguageFactory> factory) {
        string code = factory->getCode();
        factories[code] = move(factory);
    }

    const ILanguageFactory* find(const string& code) const {
        auto it = factories.find(code);
        return it == factories.end() ? nullptr : it->second.get();
    }

    vector<string> getCodes() const {
        vector<string> codes;
        for (const auto& pair : factories) codes.push_back(pair.first);
        return codes;
    }

private:
    map<string, unique_ptr<ILanguageFactory>> factories;
};

// Кинопрокат: каталог фильмов и оформление заказов
class MovieRental {
public:
    MovieRental(const LanguageRegistry& languages) : languages(languages) {}

    void addFilm(const Film& film) { catalog.push_back(film); }
    const vector<Film>& getCatalog() const { return catalog; }

    // Возвращает nullptr, если фильма или языка нет
    unique_ptr<FilmCopy> order(const string& title, const string& languageCode) const {
        const ILanguageFactory* factory = languages.find(languageCode);
        if (factory == nullptr) return nullptr;
        for (const Film& film : catalog) {
            if (film.getTitle() == title) {
                return unique_ptr<FilmCopy>(new FilmCopy(film, *factory));
            }
        }
        return nullptr;
    }

    bool changeLanguage(FilmCopy& copy, const string& languageCode) const {
```

```
        const ILanguageFactory* factory = languages.find(languageCode);
        if (factory == nullptr) return false;
        copy.setLanguage(*factory);
        return true;
    }

private:
    const LanguageRegistry& languages;
    vector<Film> catalog;
};

// ======================== Тестирование ========================

class Tester {
public:
    Tester() : passed(0), failed(0) {}

    void check(bool condition, const string& name) {
        if (condition) {
            passed++;
            cout << "  [OK]   " << name << endl;
        } else {
            failed++;
            cout << "  [FAIL] " << name << endl;
        }
    }

    void printResult() const {
        cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
    }

private:
    int passed;
    int failed;
};

void fillRegistry(LanguageRegistry& registry) {
    registry.add(unique_ptr<ILanguageFactory>(new RussianFactory()));
    registry.add(unique_ptr<ILanguageFactory>(new EnglishFactory()));
    registry.add(unique_ptr<ILanguageFactory>(new FrenchFactory()));
}

void testFactories(Tester& t) {
    RussianFactory ru;
    EnglishFactory en;
    FrenchFactory fr;
    vector<ILanguageFactory*> factories = { &ru, &en, &fr };
```

```
    bool allMatch = true;
    for (ILanguageFactory* f : factories) {
        if (f->createAudioTrack()->getLanguage() != f->getCode() ||
            f->createSubtitles()->getLanguage() != f->getCode()) {
            allMatch = false;
        }
    }
    t.check(allMatch, "Каждая фабрика создает дорожку и субтитры своего языка");
}

void testOrder(Tester& t) {
    LanguageRegistry registry;
    fillRegistry(registry);
    MovieRental rental(registry);
    rental.addFilm(Film("Интерстеллар", 2014, 169));

    unique_ptr<FilmCopy> copy = rental.order("Интерстеллар", "ru");
    t.check(copy != nullptr, "Заказ фильма");
    t.check(copy->getAudioLanguage() == "ru" && copy->getSubtitlesLanguage() == "ru",
            "Языки дорожки и субтитров совпадают при заказе");

    t.check(rental.order("Нет такого фильма", "ru") == nullptr, "Неизвестный фильм не выдается");
    t.check(rental.order("Интерстеллар", "de") == nullptr, "Неизвестный язык не выдается");
}

void testChangeLanguage(Tester& t) {
    LanguageRegistry registry;
    fillRegistry(registry);
    MovieRental rental(registry);
    rental.addFilm(Film("Интерстеллар", 2014, 169));
    unique_ptr<FilmCopy> copy = rental.order("Интерстеллар", "ru");

    t.check(rental.changeLanguage(*copy, "en") &&
            copy->getAudioLanguage() == "en" && copy->getSubtitlesLanguage() == "en",
            "При смене языка меняются и дорожка, и субтитры");
    t.check(!rental.changeLanguage(*copy, "de") &&
            copy->getAudioLanguage() == "en" && copy->getSubtitlesLanguage() == "en",
            "Смена на неизвестный язык ничего не меняет");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testFactories(t);
    testOrder(t);
    testChangeLanguage(t);
```

```
    t.printResult();
}

// ======================== Демонстрация ========================

int main() {
    setlocale(LC_ALL, "Russian");

    LanguageRegistry registry;
    fillRegistry(registry);

    MovieRental rental(registry);
    rental.addFilm(Film("Интерстеллар", 2014, 169));
    rental.addFilm(Film("Амели", 2001, 122));
    rental.addFilm(Film("Брат", 1997, 96));

    cout << "===== Демонстрация =====" << endl;
    cout << "Каталог:" << endl;
    for (const Film& film : rental.getCatalog()) {
        cout << "  " << film.getTitle() << " (" << film.getYear() << ")" << endl;
    }
    cout << "Языки:";
    for (const string& code : registry.getCodes()) {
        cout << " " << code << " (" << registry.find(code)->getName() << ")";
    }
    cout << endl << endl;

    unique_ptr<FilmCopy> film1 = rental.order("Амели", "fr");
    cout << film1->play() << endl;

    unique_ptr<FilmCopy> film2 = rental.order("Интерстеллар", "en");
    cout << film2->play() << endl;

    cout << "Меняем язык на русский:" << endl;
    rental.changeLanguage(*film2, "ru");
    cout << film2->play() << endl;

    if (rental.order("Брат", "de") == nullptr) {
        cout << "Фильм \"Брат\" на немецком недоступен" << endl;
    }

    runTests();
    return 0;
}
```